SKILLBARTER

Exhaustive Low-Level Technical Architecture & Implementation Specification

Target Evaluation: 10+ LPA SDE-1 Technical Interviews & Architecture Defense
Core Domains: Multi-Dimensional Vector Math, Intermediary Valuation Engine, Cryptography, Short Polling, Partitioned Datastores

---

1. End-to-End System Architecture & Execution Flow

SkillBarter is engineered as a decoupled, asynchronous, 3-tier web platform designed for peer-to-peer technical and creative skill bartering. The architecture completely eliminates currency-based exchange by enforcing a mathematical barter matrix balancing skill levels and categories.

1.1 Component Topology & Architecture Diagram

```
+-------------------------------------------------------------------------------------------------------+ | CLIENT
TIER (React 18 SPA / Vite) | | - SPA Routing (React Router DOM v6) - Session State & Identity (localStorage) | | -
Vector Match Cards Deck (BarterMatch.jsx) - Negotiation Terminal / Short Polling (Chat.jsx)| | - Dynamic Valuation
Calculators (Inline Math) - Warm-Dim Design System (Tailwind + CSS Vars) |
+-------------------------------------------------------------------------------------------------------+ | | HTTP /
JSON (RESTful Endpoints) v
+-------------------------------------------------------------------------------------------------------+ |
APPLICATION TIER (Node.js / Express.js) | | - Middleware Pipeline: CORS (Cross-Origin Resource Sharing) + Express
JSON Body Parser (Stream) | | - Cryptographic Subsystem: Salted PBKDF2-HMAC-SHA512 (cryptoHelper.js) | | -
Multi-Dimensional Vector Compatibility Engine: O(N) Cosine Similarity Matcher | | - Dynamic Intermediary Valuation
Engine: Udemy Category Benchmarks + Level Multipliers | | - Synchronous Data Access Object (DAO): readDb() /
writeDb() with in-memory array manipulation |
+-------------------------------------------------------------------------------------------------------+ | |
Synchronous File System I/O (Node.js fs module) v
+-------------------------------------------------------------------------------------------------------+ | STORAGE
TIER (Decoupled Partitioned JSON Datastore) | | backend/databases/ | |
+--------------------+--------------------+--------------------+--------------------+ | | | users.json |
skills.json | deals.json | messages.json | | | | (Profiles & Arrays)| (1010+ Skills) | (Barter Contracts) |
(Envelope History) | | | +--------------------+--------------------+--------------------+--------------------+ | |
| posts.json (Forum) | requests.json (Open Bulletin Board) | db.json.backup | | |
+--------------------+-----------------------------------------+--------------------+ |
+-------------------------------------------------------------------------------------------------------+
```

1.2 Complete Lifecycle of an HTTP Request

To illustrate the exact end-to-end execution flow, consider the complete trace of a client dispatching a barter proposal through **POST /api/chats/send**:

- **1. Client Dispatch (React):** The user selects an offered skill, wanted skill, level multipliers, and compensation inside Chat.jsx. The client triggers fetch(`${API_BASE_URL}/api/chats/send`, { method: 'POST', headers: {'Content-Type': 'application/json'}, body: JSON.stringify({...}) }).
- **2. Network & CORS Interception:** If origin differs, the browser issues an HTTP OPTIONS preflight request. The cors() middleware in Express intercepts the request, checks allowed origins/headers/methods, and responds with HTTP 204 No Content.
- **3. Body Parsing Stream:** express.json() captures incoming TCP packets, buffers the raw byte stream, validates JSON syntax, and populates req.body.
- **4. Route Resolution:** Express iterates through its internal regex routing table matching POST /api/chats/send.
- **5. Controller Execution & Business Logic:** The route handler extracts { from, to, text, type, deal }. If type === 'deal', the handler synthesizes a unique Deal ID: 'deal_' + Date.now() and builds an immutable deal contract entity.

- **6. Database Read & Mutation:** The DAO invokes readDb(), which executes synchronous fs.readFileSync() across the partitioned JSON files. The controller pushes newDeal to db.deals and newMessage to db.messages.
- **7. Atomic Serialization & Persistence:** writeDb(db) serializes mutated arrays to disk via fs.writeFileSync(DEALS_PATH, JSON.stringify(data.deals, null, 2), 'utf8') and MESSAGES_PATH.
- **8. Response Serialization:** The controller hydrates the message envelope with the referenced deal object and executes res.status(201).json(hydratedResponse). Client receives the response and updates its local React state array optimistically.

1.3 Detailed Repository File & Folder Responsibilities

Path / Module	Component	Architectural Responsibility & Low-Level Invariants
backend/server.js	API Controller & Gateway	Express server initialization, partitioned JSON persistence helpers (readDb/writeDb), Vector matching algorithms, Udemy valuation math, and 14 REST endpoints.
backend/cryptoHelper.js	Cryptographic Subsystem	PBKDF2-HMAC-SHA512 password hashing with 16-byte random salt generation and verification routine.
backend/databases/*.json	Storage Tier	Partitioned flat-file document collections: users.json (profiles), skills.json (1010+ catalog), deals.json (contracts), messages.json (chat history), posts.json (forum), requests.json (bulletin).
frontend/src/pages/Chat.jsx	Negotiation Workspace	Short-polling (2000ms) bidirectional messaging, barter contract drawer, dynamic client-side compensation calculator, and deal accept/decline action handlers.
frontend/src/pages/Dashboard.jsx	Dashboard & Analytics	Compatibility radar, dynamic match partner feed, profile metrics, interactive roadmaps, and fluid tablet/mobile sub-grid layout.
frontend/src/pages/BarterMatch.jsx	Discovery Swiper	Tinder-style card deck utilizing precomputed compatibility scores and active conversation exclusions.
frontend/src/config.js	Gateway Config	Dynamic API Base URL resolution supporting local dev fallback and production Render URL injection.

2. Core Technical Implementations & Algorithmic Logic

This section deconstructs the primary mathematical and programmatic algorithms powering the SkillBarter ecosystem. Every formula, time/space complexity boundary, and data transformation pipeline is rigorously analyzed.

2.1 Multi-Dimensional Cosine Similarity Vector Matching Engine

Unlike naive keyword lookups, SkillBarter projects user profiles into a multi-dimensional Euclidean vector space. The dimensionality *N* corresponds to the union of all registered skills in the platform catalog (1010+ skills). Levels are mapped to discrete scalar weights: **Easy = 1, Medium = 2, Advanced = 3**.

Mathematical Formulation:

Let **A** be the Offer Vector of User 1 and **B** be the Want Vector of User 2 in $\blacksquare^N$:
• **Vector Projection:** $\forall$ skill $s \in$ Portfolio, Vector[index(s)] = Weight(Level(s)), else 0.
• **Dot Product:** $A \cdot B = \sum_{i=1}^{N} A_i B_i$
• **Euclidean Norm (Magnitude):** $||A|| = \sqrt{(\sum_{i=1}^{N} A_i^2)}$
• **Cosine Similarity:** $\text{sim}(A, B) = (A \cdot B) / (||A|| \times ||B||) \in [0.0, 1.0]$

Bidirectional Compatibility Index:

Because bartering is a two-way reciprocal trade, directional alignment is calculated twice:
1. $sim(Other_{Offer}, Current_{Want})$: Does the candidate teach what I need?
2. $sim(Current_{Offer}, Other_{Want})$: Do I teach what the candidate needs?
Composite Score: Score = $(\text{sim}_1 + \text{sim}_2) / 2$. If $\text{sim}_1 > 0$ AND $\text{sim}_2 > 0$, matchType = **'Mutual'**; otherwise, matchType = **'Direct'**.

```
// Excerpt from backend/server.js: /api/users/matches const LEVEL_WEIGHTS = { Easy: 1, Medium: 2, Advanced: 3 }; // Calculate unique dimension space (N = 1010+ skills) const allSkillsSet = new Set(); db.skills.forEach(s => allSkillsSet.add(s.name.toLowerCase())); db.users.forEach(u => { u.offer.forEach(s => allSkillsSet.add(s.skill.toLowerCase())); u.want.forEach(s => allSkillsSet.add(s.skill.toLowerCase())); }); const allSkills = Array.from(allSkillsSet); const magnitude = (vec) => Math.sqrt(vec.reduce((sum, val) => sum + val * val, 0)); const dotProduct = (vecA, vecB) => vecA.reduce((sum, val, idx) => sum + val * vecB[idx], 0); const createSkillVector = (skillList) => { const vec = new Array(allSkills.length).fill(0); skillList.forEach(item => { const idx = allSkills.indexOf(item.skill.toLowerCase()); if (idx !== -1) vec[idx] = LEVEL_WEIGHTS[item.level] || 1; }); return vec; }; // Execution over candidate population: const uOfferVec = createSkillVector(currentUser.offer); const uWantVec = createSkillVector(currentUser.want); // Loop over all candidate users O(U * N)...
```

Asymptotic Complexity Analysis:

• **Time Complexity:** Let *U* be total users, *S* be skills per user (|S| ■ 10), and *N* be total skills catalog size (1010). Vector generation takes *O(|S|)* using hash lookups. Computing dot product and magnitude across dense arrays takes *O(N)*. Total matching cost for *U* candidates is **O(U × N)**. Sorting results takes *O(U log U)*.
• **Space Complexity:** Storing the *N*-dimensional dense array per user requires **O(N)** heap allocation. For 10k users, dense vectors require substantial memory; sparse dictionary vector representation *O(|S|)* is the recommended remediation.

2.2 Dynamic Intermediary Barter Pricing Valuation Engine

To resolve the fundamental economic challenge of bartering (the 'Coincidence of Wants' problem and value asymmetry), SkillBarter acts as an algorithmic financial intermediary. It assigns market valuations inspired by Udemy professional course benchmarks.

Category	Base Price (■)	Level	Multiplier	Effective Valuation (■)
Technology	■450	Easy / Med / Adv	1.0x / 1.5x / 2.5x	■450 / ■675 / ■1,125
Business	■400	Easy / Med / Adv	1.0x / 1.5x / 2.5x	■400 / ■600 / ■1,000
Design	■350	Easy / Med / Adv	1.0x / 1.5x / 2.5x	■350 / ■525 / ■875
Marketing	■300	Easy / Med / Adv	1.0x / 1.5x / 2.5x	■300 / ■450 / ■750
Photography / Music	■250	Easy / Med / Adv	1.0x / 1.5x / 2.5x	■250 / ■375 / ■625
Languages	■200	Easy / Med / Adv	1.0x / 1.5x / 2.5x	■200 / ■300 / ■500

Algorithmic Valuation Formula:

Value(Skill, Level) = BasePrice[Category(Skill)] × Multiplier[Level]
Net Compensation (Δ) = ∑ Value(Taught by Alice) - ∑ Value(Taught by Bob)
• **Nullification Condition (Δ = 0):** When skill values cancel out (e.g., React Medium ■675 vs Python Medium ■675), cash adjustment is strictly ■0 (Balanced Trade).
• **Asymmetric Trade (Δ > 0):** Alice's skill holds higher market value; Bob is obligated to compensate Alice with ■Δ.
• **Asymmetric Trade (Δ < 0):** Bob's skill holds higher market value; Alice is obligated to compensate Bob with ■|Δ|.
Bargaining Freedom: The system auto-fills the calculated recommendation in the UI drawer, but allows custom bargaining overrides.

2.3 Cryptographic Authentication Subsystem

Credential storage enforces salted password cryptography using Node's native crypto.pbkdf2Sync. It generates a cryptographically secure pseudorandom 16-byte salt via CSPRNG (crypto.randomBytes(16)). The derivation executes 1,000 iterations of HMAC-SHA512 producing a 64-byte key length (128 hex chars). This completely eliminates precomputed rainbow table attacks and credential leakage vulnerabilities.

3. Database Design, Schemas & Query Patterns

SkillBarter utilizes a partitioned JSON document storage architecture located in backend/databases/. Data is organized into distinct entity domains to isolate disk write locks and optimize read patterns.

3.1 Complete Schema Specification

```
1. users.json (Entity: User Profile) { "email": "String (Primary Key, lowercased, unique, RFC 5322 format)", "name": "String (Display name, 2-50 chars)", "salt": "String (32-char hex, 16 CSPRNG bytes)", "passwordHash": "String (128-char hex, PBKDF2-HMAC-SHA512 key)", "offer": [ { "skill": "String", "level": "Easy|Medium|Advanced", "category": "String" } ], // Embedded array "want": [ { "skill": "String", "level": "Easy|Medium|Advanced", "category": "String" } ], // Embedded array "bio": "String (Sanitized plain text, max 500 chars)", "rating": "Float (1.0 to 5.0, single decimal precision)", "completedExchanges": "Integer (Non-negative counter)", "badges": [ "String (e.g., 'First Barter', 'Top Mentor')" ], "reviews": [ { "id": "String", "reviewerName": "String", "reviewerEmail": "String", "rating": "Float", "comment": "String", "date": "String" } ] } 2. deals.json (Entity: Barter Contract) { "id": "String (Primary Key, 'deal_' + EpochTimestamp)", "proposer": "String (Foreign Key -> users.email)", "receiver": "String (Foreign Key -> users.email)", "skillOffered": "String (Skill name)", "skillOfferedLevel": "Easy|Medium|Advanced", "skillWanted": "String (Skill name)", "skillWantedLevel": "Easy|Medium|Advanced", "compensation": "Float (Monetary adjustment, 0.0 for balanced trade)", "status": "pending | accepted | declined" } 3. messages.json (Entity: Communication Envelope) { "id": "String (Primary Key, 'msg_' + EpochTimestamp)", "from": "String (Sender email or 'system')", "to": "String (Recipient email)", "text": "String (Body content)", "type": "text | deal | system", "dealId": "String | null (Foreign Key -> deals.id)", "systemFor": [ "String" ] | undefined (Dual-visibility broadcast marker), "timestamp": "String (HH:MM format)" }
```

3.2 Embedding vs. Referencing Strategy

SkillBarter adopts a hybrid schema strategy balancing denormalization and relational integrity:
• **Embedded Arrays:** Skills (offer, want), badges, and reviews are embedded directly inside users.json. *Justification:* User skill portfolios are bounded collections ($|S| \leq 20$). Embedding guarantees single-read profile retrieval without performing costly relational joins or multi-file disk I/O.
• **Relational Referencing:** deals.json and messages.json reference users via email keys (proposer, receiver, from, to). Messages reference deals via dealId. *Justification:* Messages and deals grow unboundedly over time (1:N relationship). Embedding messages within user documents would cause unbounded document bloat and memory exhaustion.

3.3 Concurrency, Read/Write Patterns & Mutation Handling

Node.js executes JavaScript on a single-threaded Event Loop. Therefore, in-memory array operations are inherently thread-safe and non-preemptive. However, disk persistence via writeDb() uses synchronous file system writes (fs.writeFileSync). Under high concurrency, parallel requests executing readDb() and writeDb() can suffer from **Lost Update Anomaly**: Request A reads file; Request B reads file; Request A mutates and writes; Request B mutates and overwrites, clobbering Request A's write. *Interview Defense Remediation:* Implement an in-memory queue or replace with ACID document transactions in MongoDB / PostgreSQL with row-level locks.

4. Engineering Trade-offs & Justifications (Interview Defense)

Every architectural decision involves deliberate trade-offs. Below is the technical defense justifying the chosen stack against industry alternatives:

Decision Point	Selected Option	Alternative	Technical Justification & Accepted Trade-offs
Backend Engine	Node.js / Express	Java Spring Boot / Go	**Advantage:** Event-driven non-blocking I/O, native JSON serialization without reflection overhead, and uniform JavaScript typing across client/server. **Trade-off Accepted:** Single-threaded CPU limitations during dense vector math computation.
Persistence Layer	Partitioned JSON Files	PostgreSQL / MongoDB	**Advantage:** Zero external infrastructure dependencies, instantaneous bootstrap, 100% portable file state, zero cold-start database connection overhead. **Trade-off Accepted:** Lack of B-Tree indexing (O(N) table scans) and lack of ACID multi-file write transactions.

Messaging Engine	Short Polling (2000ms)	WebSockets / Socket.io	**Advantage:** Completely stateless horizontal scaling, zero persistent socket connection pool state, seamless firewall/proxy traversal, HTTP/2 multiplexing compatibility. **Trade-off Accepted:** Higher network header bandwidth overhead and an average message synchronization latency of ~1000ms.

5. Scale Bottlenecks, Failure Modes & Resilience (10k Concurrent Users)

Subjecting SkillBarter to 10,000 concurrent active users reveals distinct computational, I/O, and memory bottlenecks. Below is an exhaustive identification of Single Points of Failure (SPOF) and concrete production remediation strategies:

5.1 Identification of Bottlenecks & Failure Modes

- **1. Event Loop Starvation (CPU-Bound):** crypto.pbkdf2Sync executes 1,000 SHA-512 iterations synchronously on the main Event Loop thread. Under 100 simultaneous authentication requests, the event loop freezes for hundreds of milliseconds, queuing all network I/O and causing severe cascading latency.
- **2. Dense Vector Space Computation (CPU-Bound):** Matching a user against 10,000 candidate profiles across an N=1010 dense vector space requires $10{,}000 \times 1010 = 1.01 \times 10^7$ floating-point operations per request. Under concurrent match requests, CPU utilization saturates at 100%.
- **3. Disk I/O & IOPS Saturation (I/O-Bound):** readDb() and writeDb() perform synchronous file reading and JSON parsing (JSON.parse / JSON.stringify) on every write operation. Disk throughput becomes a hard bottleneck, leading to file access contention and write latency spikes.
- **4. Polling Multiplication Spike:** 10,000 concurrent chat users polling every 2,000ms generate 5,000 HTTP requests per second (RPS) directly hitting the Express API gateway.

5.2 Production Architectural Remediation Roadmap

Vulnerability	Root Cause	Architectural Remediation (Production Blueprint)
PBKDF2 Event Loop Freeze	Synchronous CPU execution on main thread	Migrate to asynchronous crypto.pbkdf2 delegating derivation to the libuv background worker pool. Configure UV_THREADPOOL_SIZE=64.
O(U × N) Vector Overhead	Dense array dot product over full catalog	1. Migrate to PostgreSQL using **pgvector** extension with HNSW (Hierarchical Navigable Small World) index for sub-millisecond approximate nearest neighbor (ANN) vector search. 2. Or implement sparse vector representations: only store non-zero skills in hash maps.
Disk Write Contention	Synchronous fs.writeFileSync	Migrate to MongoDB/PostgreSQL with connection pooling (e.g. pg-pool / HikariCP). Introduce **Redis Cache-Aside**: cache user profile vectors and skills catalog with TTL invalidation.
5,000 RPS Polling Load	2000ms short polling interval	1. Transition from Short Polling to **WebSockets (ws / socket.io)** backed by Redis Pub/Sub for horizontal message fanout. 2. Deploy an AWS Application Load Balancer (ALB) distributing traffic across auto-scaling ECS / Kubernetes Docker containers with Nginx rate limiting (100 req/min/IP).

6. Edge Cases, Security & Defensive Architecture

A robust production system must anticipate adversarial input and defensive boundary failures. Below is an audit of security vectors and edge case protections implemented in SkillBarter:

6.1 Security Vector Defense

• **Rainbow Table Attack Immunity:** Passwords are never hashed plainly. 16-byte random salts ensure two users with identical passwords yield completely disparate 128-character hashes.
• **Cross-Site Scripting (XSS) Defense:** React's Virtual DOM treats all interpolated values as raw strings by default, neutralizing injected <script> tags. The application strictly avoids dangerouslySetInnerHTML.
• **NoSQL / SQL Injection Immunity:** Because queries are executed via native JavaScript array methods (find, filter, some) operating on sanitized memory objects, SQL and NoSQL operator injection vectors (e.g. $gt: '') are inherently neutralized.
• **Cross-Site Request Forgery (CSRF) Posture:** Session identity is stored in localStorage and transmitted via explicit payload bodies or headers rather than ambient browser cookies. This makes CSRF attacks ineffective.

6.2 Edge Cases Handled vs. Unhandled Deficiencies

Scenario / Edge Case	Status	Engineered Behavior & Defensive Handling
Zero-Vector Magnitude (||v|| = 0)	HANDLED	If a user has 0 offers or 0 wants, magnitude is 0. Code checks uOfferMag > 0 && oWantMag > 0, safely returning similarity score 0 instead of NaN / division by zero.
Self-Trading & Self-Messaging	HANDLED	Filters explicitly exclude self: if (otherUser.email.toLowerCase() === currentUser.email.toLowerCase()) return;
Case & Whitespace Invariance	HANDLED	All emails and skill keys are systematically transformed via .toLowerCase().trim() to prevent duplicate accounts and missed matches.
Concurrent Barter Settlement	UNHANDLED	If two users accept conflicting barter proposals simultaneously, lack of 2-Phase Commit (2PC) or row locks could allow a user to trade the same skill slot twice.
Schema Validation Enforcement	PARTIAL	Basic presence checks exist (if (!email) ...), but missing comprehensive validation against malformed payloads. Remediation: integrate **Zod** schemas on Express routes.

End of Technical Specification — SkillBarter Architecture Reference Manual